AIN SHAMS UNIVERSITY-

FACULTY OF ENGINEERING

[MCT211s]

Automatic Control

Final Submission Report

Team 16

Name	ID	Section
Abdullah Mahmoud Abdelkarim	2100658	1 MCT
Abdelkarim Wael Abdelkarim	2101733	1 MCT
Mohamed Islam Salah Aldin Bodour	2101439	1 MCT
Musa Gamal El-Shenawy	2100614	1 MCT
Zeyad Samer Lotfy	2100820	1 MCT

Provided to:

Dr. Diaa Emad Abdel Fattah Mohamed

Table of Contents

Table Of Figures

Introduction

Precise motion control is a cornerstone of modern automation, robotics, and mechatronic systems. This project focuses on the development and implementation of a 2 Degree-of-Freedom (2-DOF) mechanism designed to achieve accurate position and speed control through a closed-loop feedback system. Leveraging the power of MATLAB/Simulink along with microcontroller programming support packages, this project integrates simulation and physical implementation to bridge theoretical control design with real-world performance.

The system is designed to utilize PID (Proportional-Integral-Derivative) control algorithms, which are tuned for both position and velocity regulation. The control design is iteratively tested through simulations, allowing for the evaluation and refinement of controller parameters based on response characteristics such as rise time, settling time, steady-state error, and stability under varying operational conditions.

Following successful simulation, the controller is deployed to a physical model using a microcontroller, enabling real-time execution of the control logic. Performance of the physical system is then compared to the simulation results to validate accuracy and effectiveness. This comprehensive approach provides valuable insights into the design, tuning, and real-world implementation of feedback-controlled mechanical systems.

Through this project, students gain hands-on experience with system modeling, controller design, dynamic simulation, and embedded implementation—key competencies in control systems engineering.

Block diagram For the System

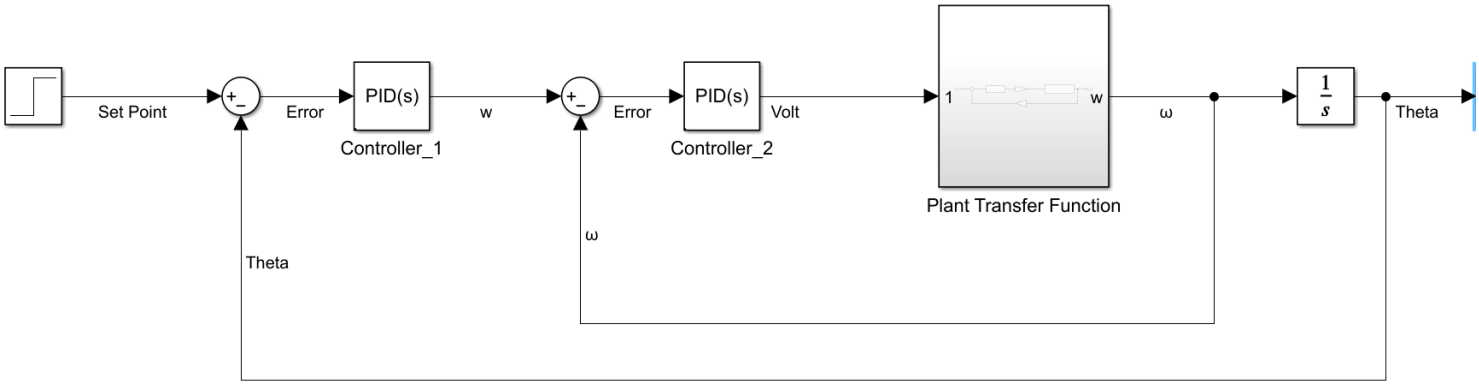

Figure 1 Block diagram For the System

System Components:

➢ Set Point

- This is the desired angular position (Theta) input, typically a step input in control systems.

➢ Outer Loop: Position Control

- Error = Set Point - Theta: The difference between the desired and actual position.
- Controller_1 (PID(s)): A PID controller that generates a velocity command (w) based on the position error.

➢ Inner Loop: Velocity Control

- Error : The difference between the desired velocity and the actual velocity.
- Controller_2 (PID(s)): Another PID controller that generates a control signal (voltage) for the plant based on velocity error.

➢ Plant Transfer Function

- Represents the physical system or plant (e.g., a DC motor).
- Takes in voltage and outputs angular velocity (ω).

➢ Integrator (1/s)

- Integrates angular velocity (ω) to obtain angular position (Theta).

➢ Feedback Loops

- Outer loop feedback: Theta is fed back to the first summing junction for position error calculation.
- Inner loop feedback: ω is fed back to the second summing junction for velocity error calculation.

Control Strategy

This is a cascaded control architecture, where:

- The outer loop ensures position tracking (slower loop).
- The inner loop ensures velocity control (faster loop).
  This structure improves response time and robustness.

System Model Using Simulink Block

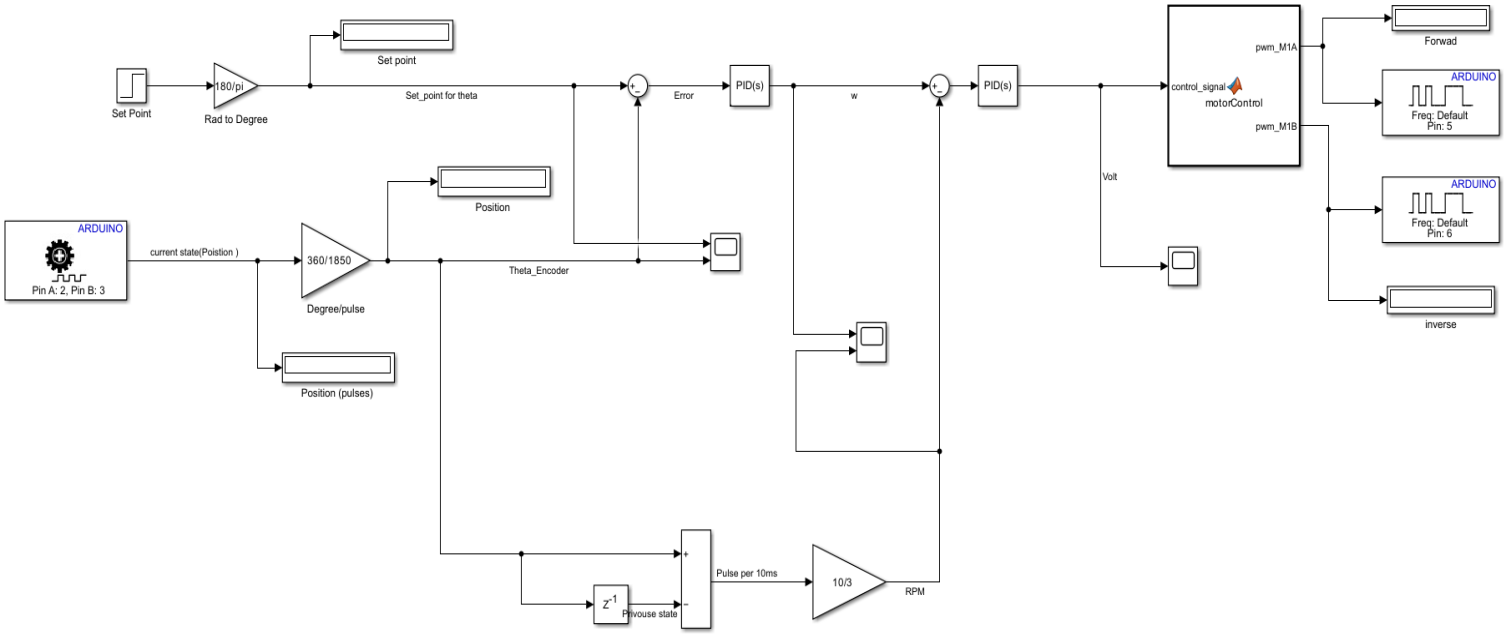

Figure 2 System Model Using Simulink Block

Set Point

- Determine the set point(Rad) and convert it to degree

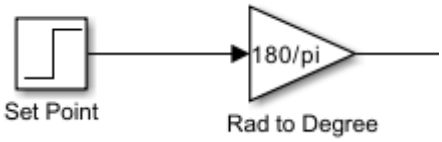

Figure 3 Set Point

Position

- Reading the actual value of the position using encoder and convert the number of pulses to degrees

Figure 4 Position Reading

Velocity

- convert the number of pulses to RPM

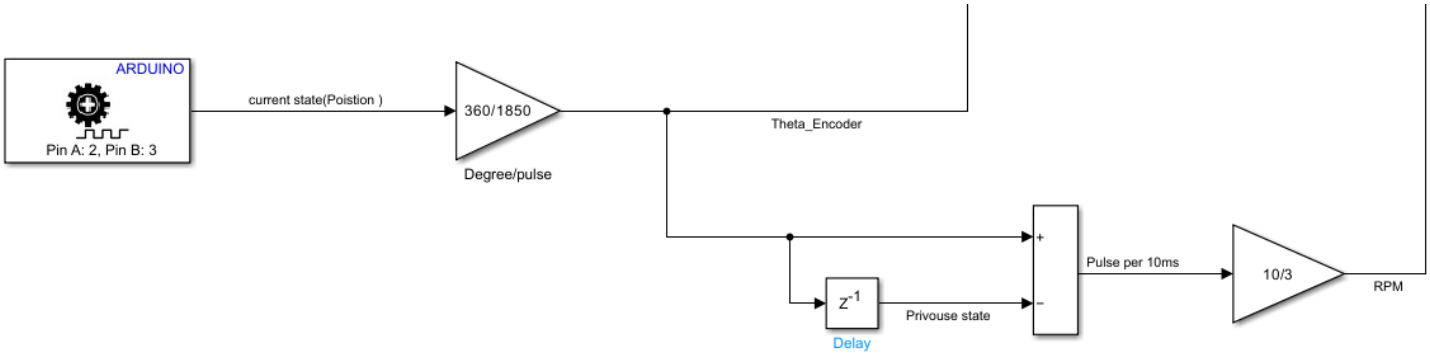

Figure 5 Velocity Reading

Position Control

By getting the error between the set point and the actual position, the PID convert the error to the angular velocity desired.

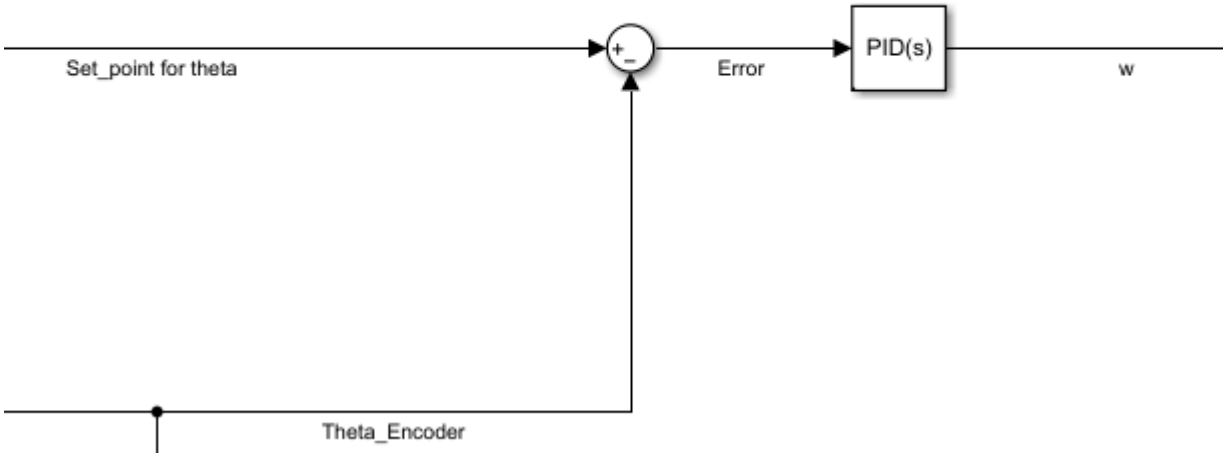

Figure 6 Position Control

Velocity Control

By getting the error between the angular velocity and the desired Velocity, the PID convert the error to PWM voltage.

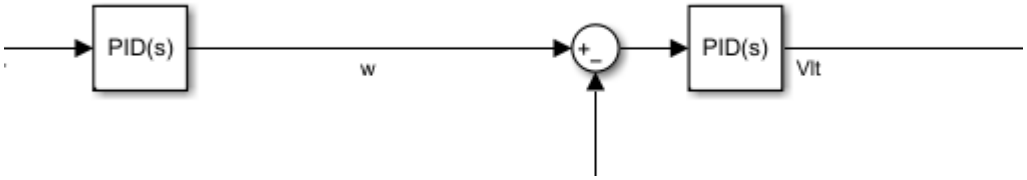

Figure 7 Velocity Control

Motor driver function

- link the Arduino to the Simulink

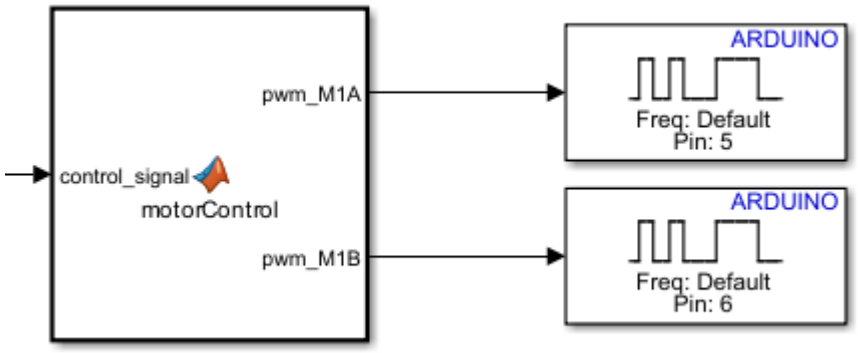

Figure 8 Motor driver function Block

```
function [pwm_M1A, pwm_M1B] = motorControl(control_signal)
%#codegen

% Constants
MAX_PWM = 255;
MIN_PWM = -255;

% Clamp control signal
control_signal = max(min(control_signal, MAX_PWM), MIN_PWM);

% Initialize outputs
pwm_M1A = 0;
pwm_M1B = 0;

% Determine direction and PWM
if control_signal > 0
    pwm_M1A = 0;  % LOW (Forward)
    pwm_M1B = control_signal;

elseif control_signal < 0
    pwm_M1A = abs(control_signal);  % HIGH (Reverse)
    pwm_M1B = 0;

else
    % Already initialized to 0 (lock)
end
```

Figure 9 Motor driver function

Simscape Model

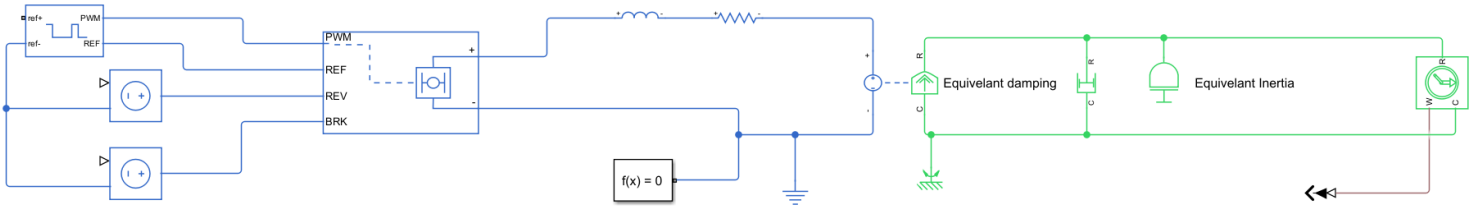

Figure 10 Simscape model

Electrical Subsystem

PWM Signal Generator & H-Bridge Driver

- Top-left: A reference signal (ref+) and (ref−) is used to generate PWM.
- Drives an H-Bridge (DC-DC converter) for motor control.

H-Bridge Block

- Receives PWM and control signals (REF, REV, BRK).
- Outputs a DC voltage to the motor circuit.
- This is the power electronics interface between controller and motor.

Armature Circuit

- Contains:
  - Inductance (L): Motor armature inductance.
  - Resistance (R): Armature winding resistance.
- Captures electrical dynamics of the motor.

Voltage Sensor or Measurement Node

- Positioned before connecting to the mechanical domain (torque output).

Mechanical Subsystem

Rotational Mechanical Interface

- Interface block connecting the electrical torque output to mechanical motion.

Equivalent Damping

- Models viscous friction or damping in the mechanical load.

Equivalent Inertia

- Models rotational inertia of the motor and load.

Rotational Motion Sensor

- Measures angular velocity or position.
- Feeds back to the controller for closed-loop control.

This system simulates a realistic electromechanical model of a DC motor including switching behavior (PWM), physical components, and mechanical dynamics.

Electronics and control system documentation

This circuit is designed to control multiple motors and servos using an Arduino Mega 2560 microcontroller. It includes JGY-370 And JGA25-371 DC Geared motors with encoders, two MG996R servos, one MG90S servo, and an L298N motor driver. Power is supplied by a 12V adapter OR a 12V lithium-ion battery. A LM7805 MOSFET is used to control the power to the motor driver.

Microcontroller Integration and Circuit Schematics

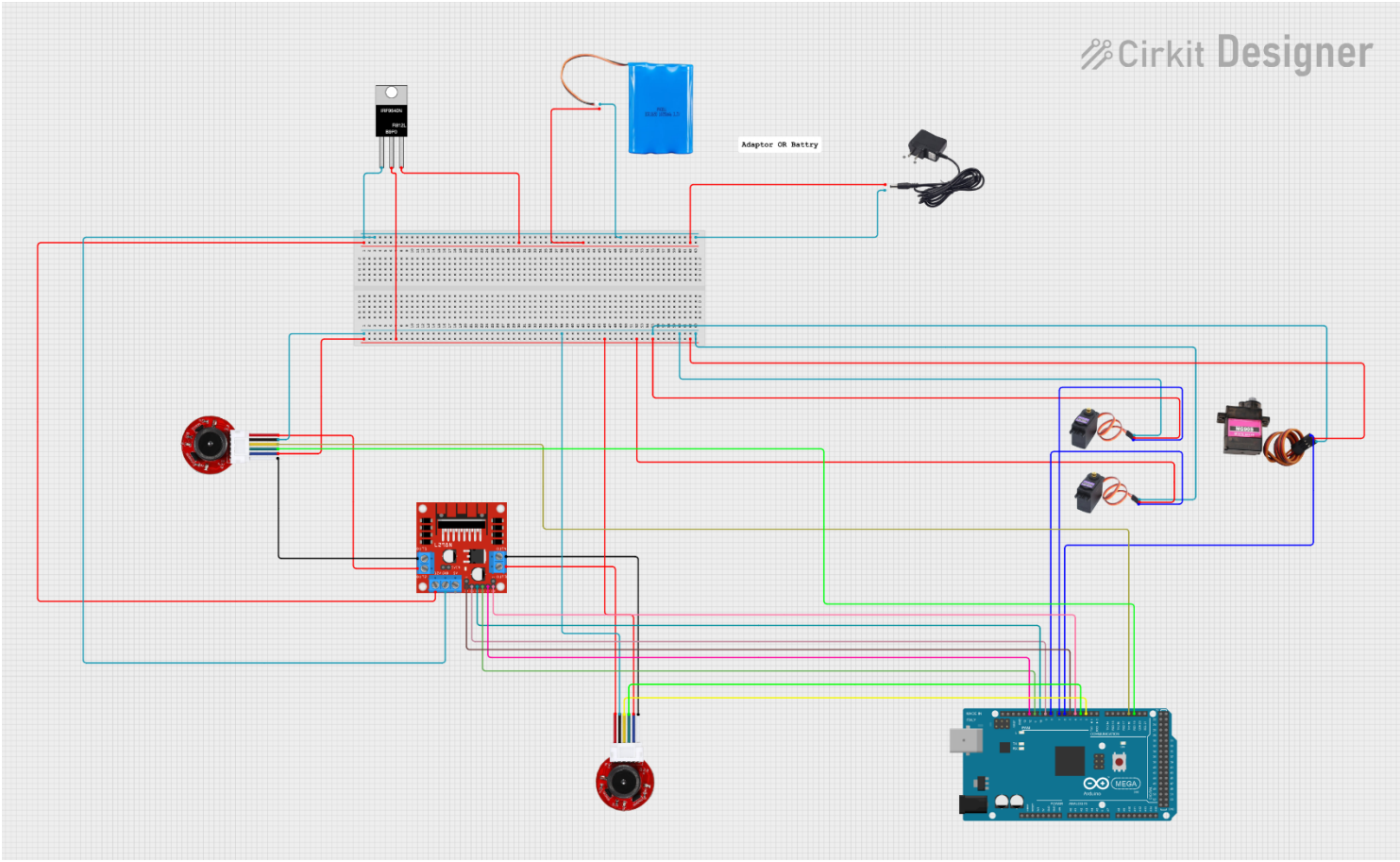

Figure 11 Robot Motion Control System Circuit

FULL PIN MAPPING SUMMARY

Device	Arduino Pin
Motor A IN1/IN2	D9, D10
Motor B IN3/IN4	D11, D12
ENA/ENB (PWM)	D5, D4
Encoder A (motor A)	D2 (INT)
Encoder B (motor A)	D3
Encoder A (motor B)	D18 (INT)
Encoder B (motor B)	D19
Servo MG996R #1	D6
Servo MG996R #2	D7
Servo MG90S	D8

Hint:

We use cytron MDD3A motor driver as we need linear voltage to PWM relation and more delivered current.

DC Motor Position Control Using PID with Encoder Feedback

To control the position of two DC motors using a PID controller based on encoder feedback, ensuring stability, minimal overshoot, and accurate steady-state response.

Experimental Procedure

➢ System Initialization

➢ Initial Testing

- Applied a step input (desired position) to the motors. Observed open-loop response: unstable, underdamped, high overshoot, and slow settling.

➢ Tuning Method: Trial and Error

- Started with all PID gains set to zero.
- Increased Kp incrementally:
  - System started to respond to input but was unstable and oscillated.
  - Increases speed of response.
  - High Kp reduces rise time but causes overshoot.
- Introduced Ki to eliminate steady-state error:
  - Response improved over time.
  - But too high Ki caused instability or integral windup.
  - Eliminates steady-state error.
- Added Kd to reduce oscillations:
  - Improves stability and damping.
  - Reduces overshoot and **oscillation**, but sensitive to noise.
- Add LPF filter:
  - Filters high-frequency noise in encoder signal to prevent noisy derivative action.

Final PID Parameters

The best performance (minimal overshoot, fast response, and zero steady-state error) was achieved with the following values:

- Underdamped response with acceptable overshoot (~5–10%).
- Settling time decreased significantly.
- Proper tuning of PID gains significantly improved control performance.
- The trial-and-error approach was effective but time-consuming.

Final Tuning Parameter

Tuning For Motor 1

Yellow : Actual position

Red : Target

Black: Control Action

- We designed the system with little overshoot for small friction in motion.

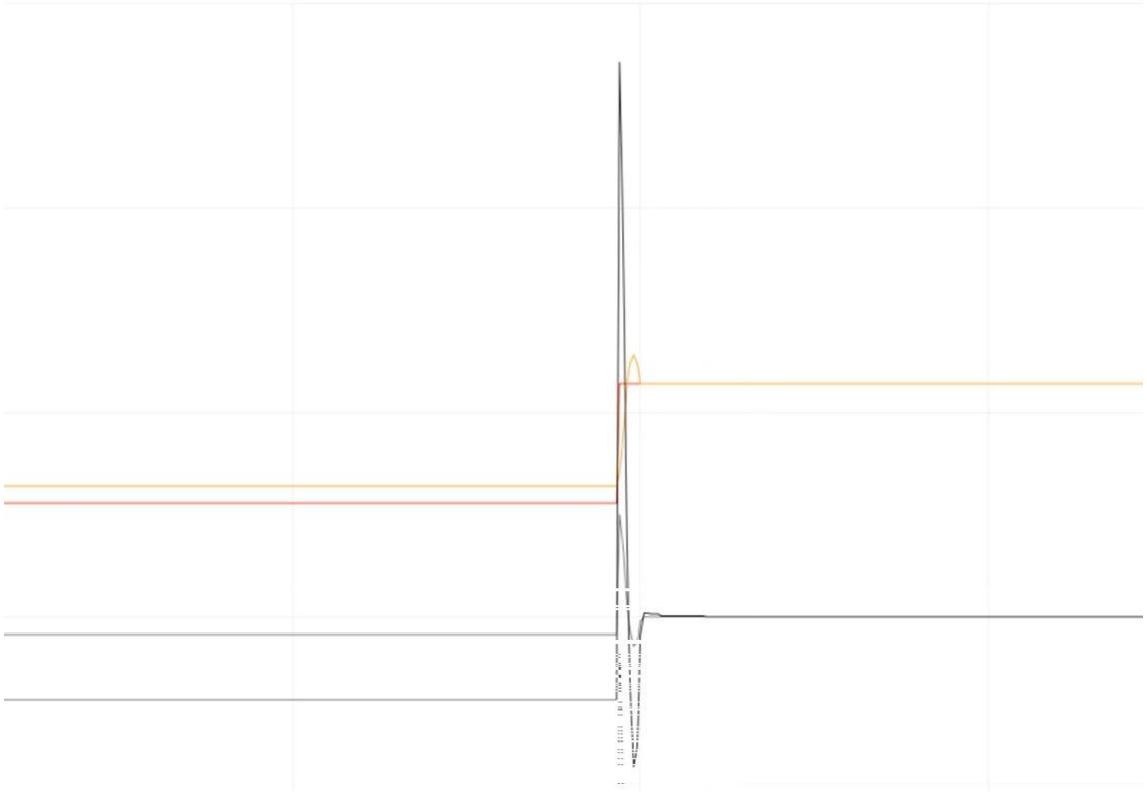

Figure 12 Response of Motor 1

	Kp	Ki	Kd
M1	5	0.4	0.3
M2	2.2	0	0.8

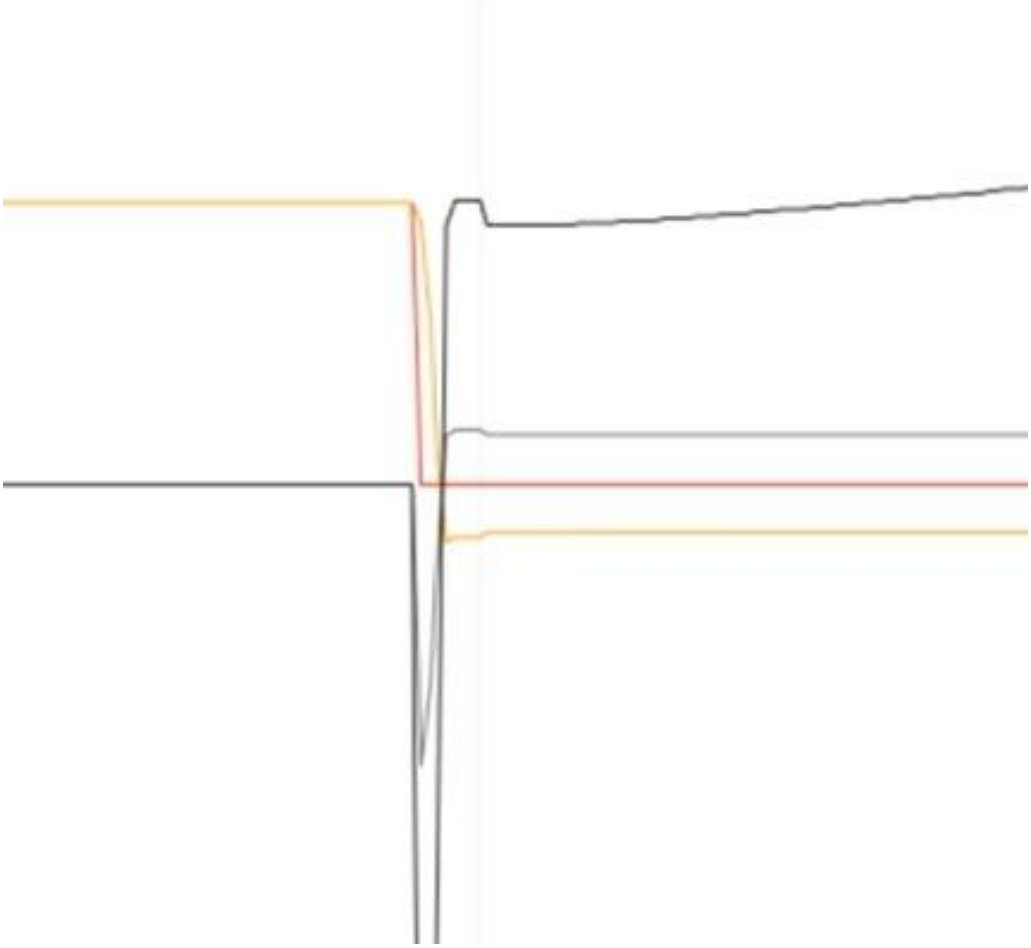

Figure 13 Response of Motor 1

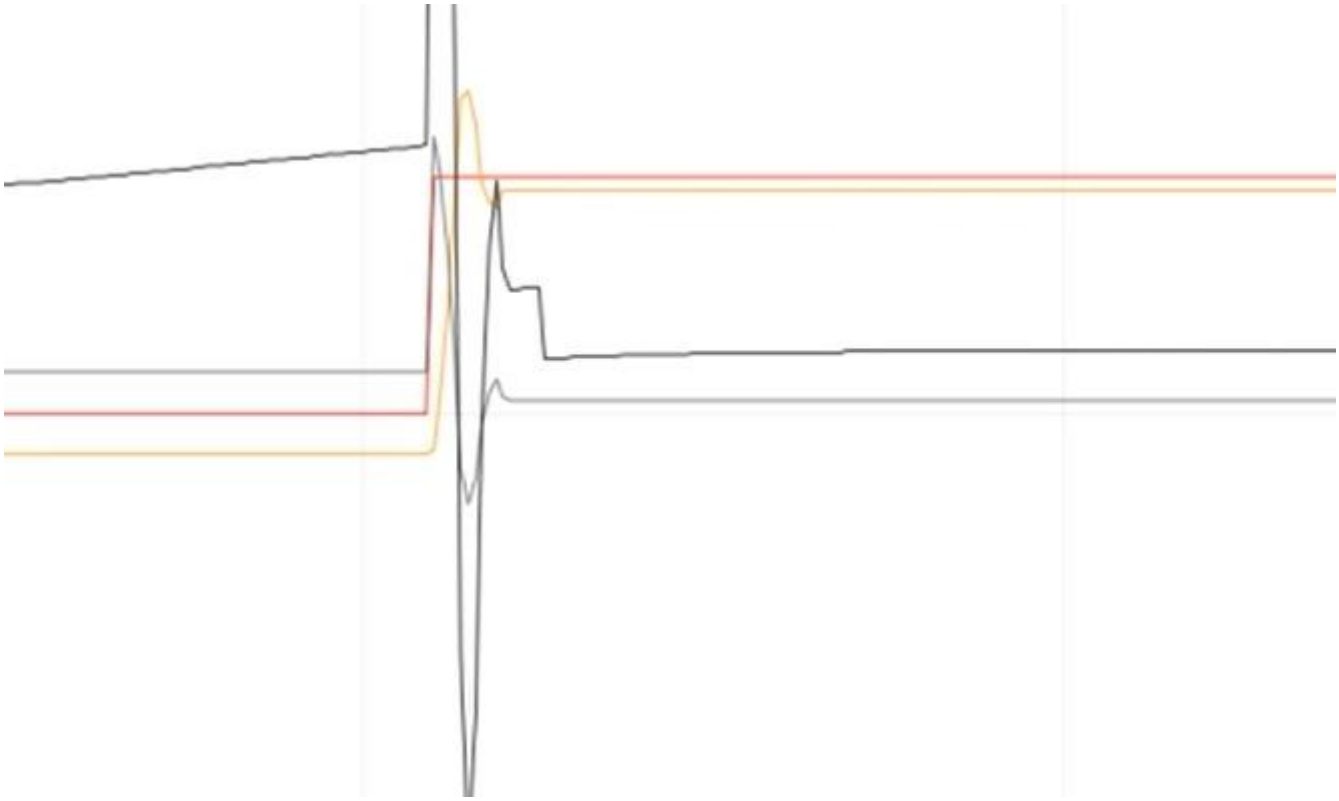

Figure 14 Response of Motor 1

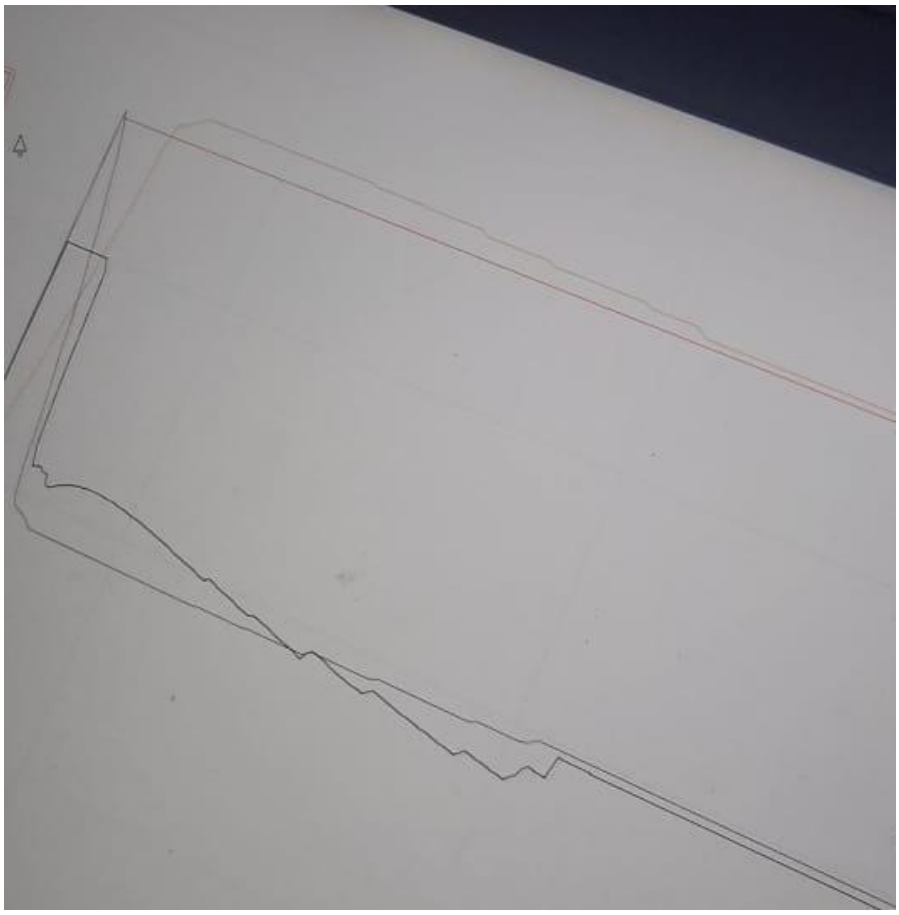

Figure 15 Response of Motor 2

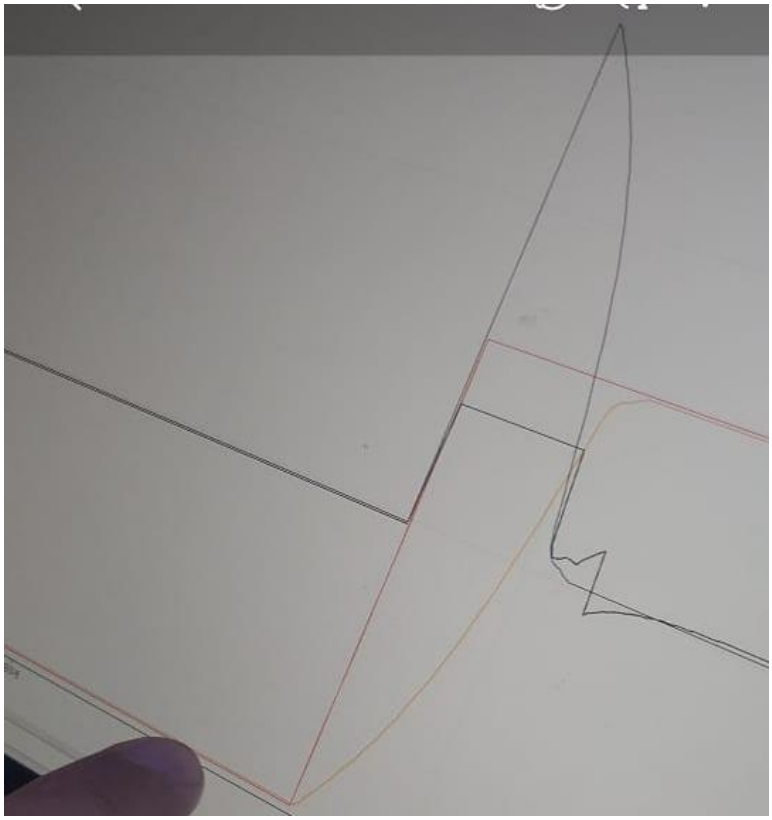

Figure 16 Response of Motor 2

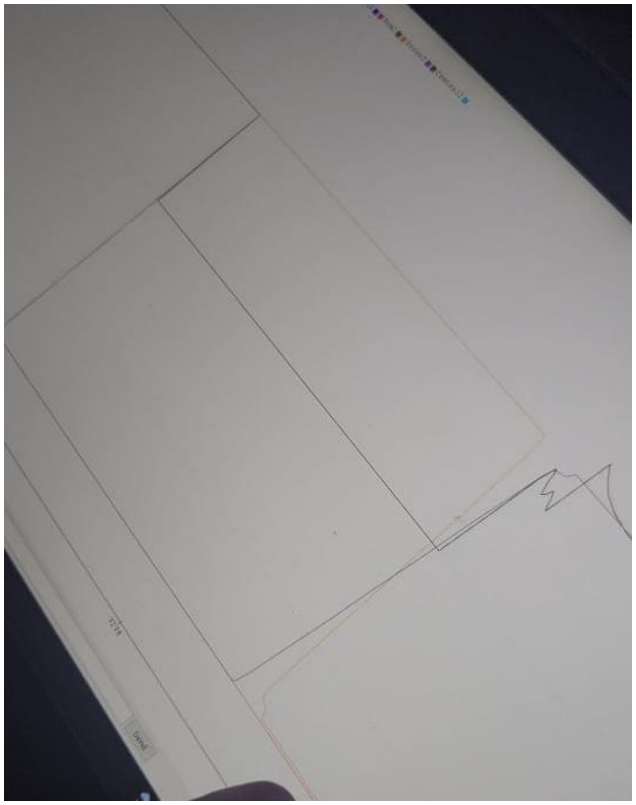

Figure 17 Response of Motor 2

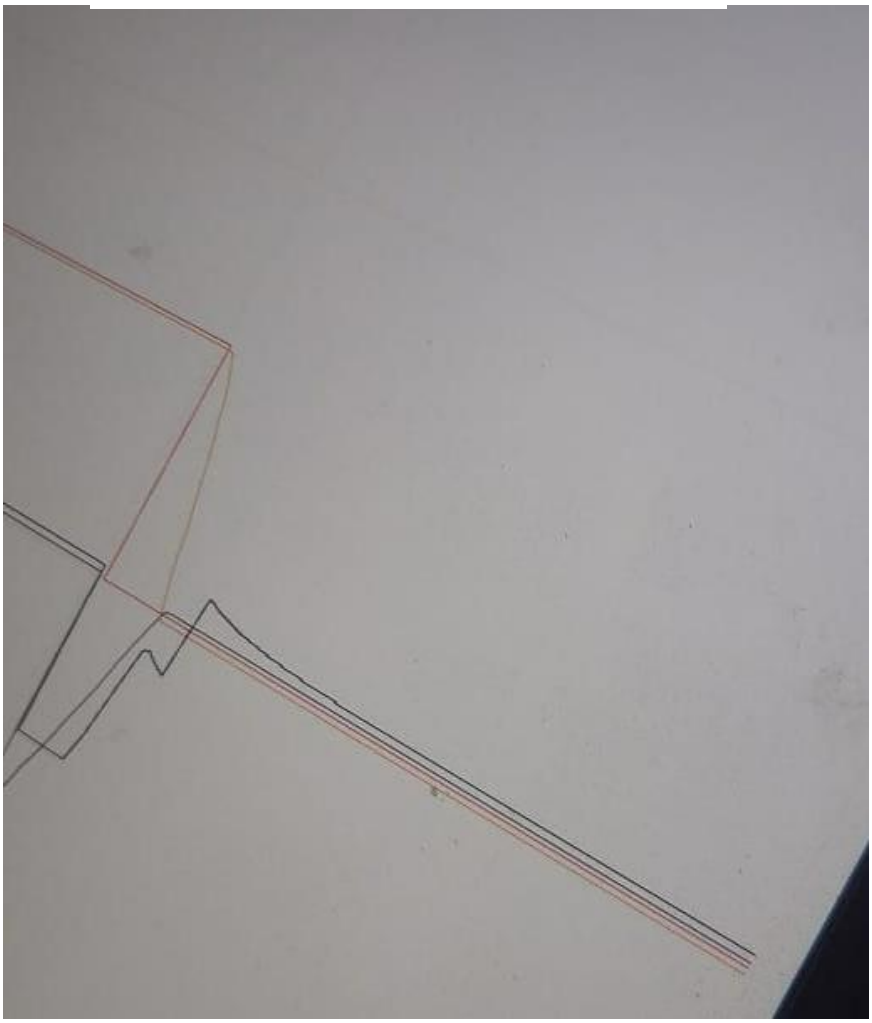

Figure 18 Response of Motor 2

Mathematical Model of the System:-

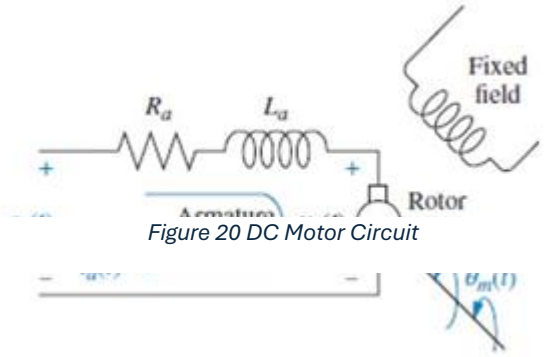

Figure 20 DC Motor Circuit

1-Electrical equation:-

$$Ea=(Ra+LS)Ia(s)+ Kb\,\omega$$

$$Vin=Ea$$

2-Mechanical model

$$J\dot{\omega} = Tm - Tdamping - Tstatic \rightarrow \text{Neglecting initial condition(static torque)}$$

$$Tm = kt \times Ia(s)$$

$$Tdamping = Beq \times \omega$$

$$JeqS\,\omega = kt \times Ia(s) - Deq \times \omega$$

$$Deq=Dm+Dmech.sys$$

$$Jeq=Jm+Jmech.sys$$

Transfer Function:-

$$\frac{\omega}{Vin} = \frac{1}{\frac{R}{Kt}(S^2 Jeq + DeqS) + SKb} \rightarrow \text{For simplicity we neglect L}$$

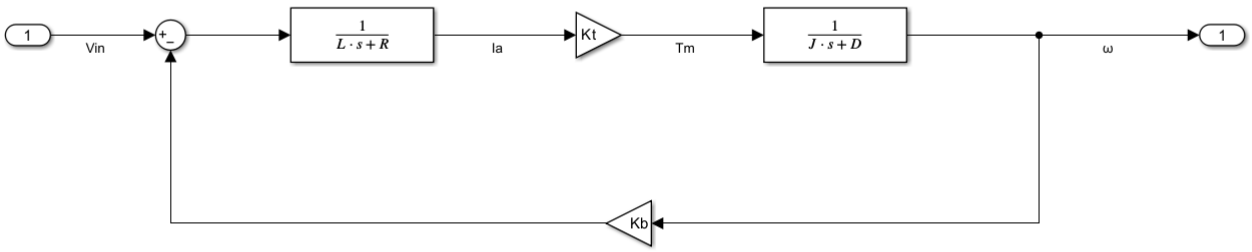

Figure 21 Block Diagram Model Of The System

Readings From the System

A block diagram for Taking Reading from the real system

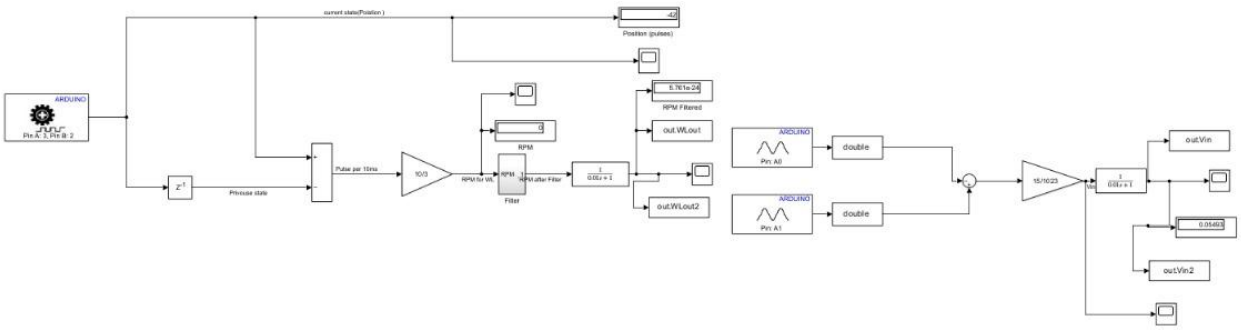

Figure 22 Block Diagram For Motor and Encoder

- We made a voltage divider circuit to measure the voltage across the motor through the Arduino Analog pins
- we divided the voltage across the motor by 3 to be able to measure it through the Arduino safely As max vin for pins =5 V

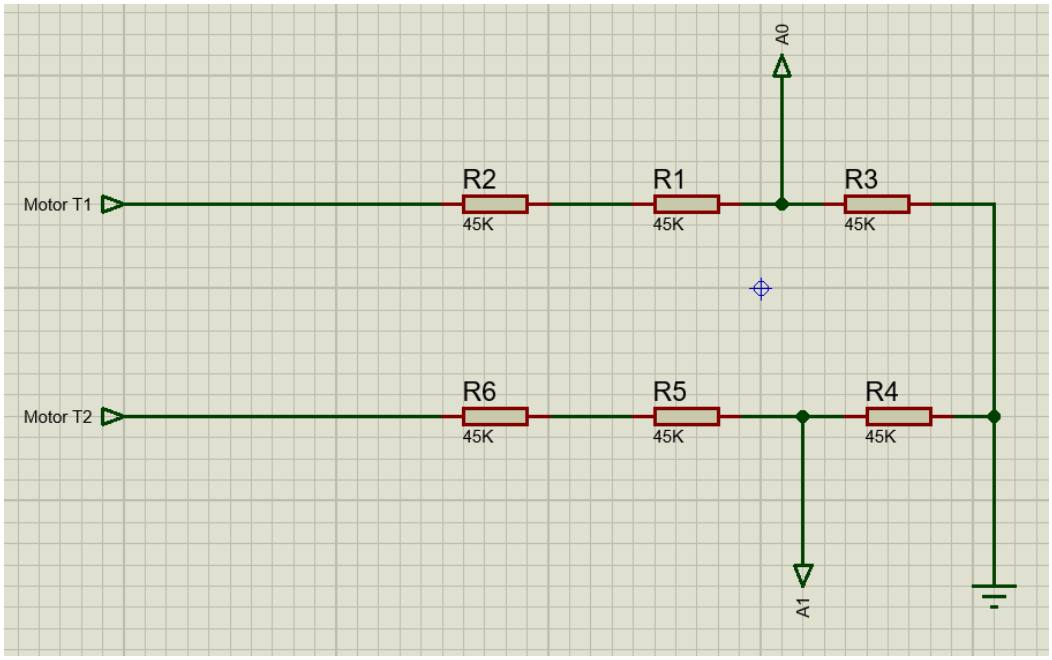

Figure 23 Voltage Divider Circuit

Parameter Estimation Procedures : -

-We need to make a parameter estimation for R,Kt,Kb,L,D and J

-the system is 2DOF so we could Estimate 3 Parameter in 1 Experiment

So we separate the Parameter estimation to 2 Steps :-

1-Motor Estimation

To estimate R , L , Kb,Kt ,D And J of the Motor

Considering the energy conversion

Pm=Tm ω=Pe=e ia→kT ia ω=ke ω ia

kT=ke=k

that's why we assume that kT=ke=k

- we take a 4 Readings from our system to estimate the R and K
  we solve it and it gives us R and K with an Error 3.64%
  From Equation Vin = $\mathrm{RI}+\mathrm{L}\frac{di}{dt}+\mathrm{k}\omega$

  At steady state Vin = $\mathrm{RI}+\mathrm{k}\omega$

K	R	Vin	current	N	ω	R I +kω	RI+kω-Vin	(RI+kω-Vin)^2	((RI+kω-Vin)/Vin)x100
0.499	4.8	10.3	0.1275	186	19.47787	10.35169	0.051691	0.002672	0.501857
		8	0.115	142	14.87021	7.98779	-0.01221	0.000149	-0.15262
		6	0.097	104	10.89085	5.911612	-0.08839	0.007812	-1.47314
		1.5	0.062	24	2.513274	1.554623	0.054623	0.002984	3.64153
							Sum	0.013617	

Figure 24 Estimation for R and K

- From this Estimation we take R=4.8 ohm and K=0.5

- We build motor Model at figure 16 To estimate L,D,J

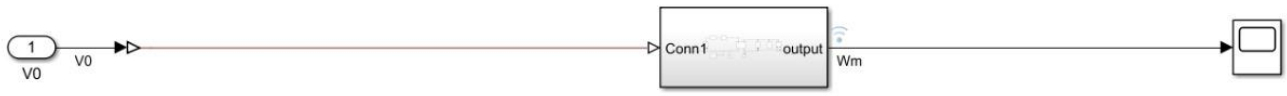

Figure 25 Motor Model Subsystem

From the Estimation with 10 iterations and validation we got that

Dm= 0.0044698 N.m/(rad/s)

L=4.38 mH

Jm=0.0081555 $Kg.m^2$

Transfer function of the block:-

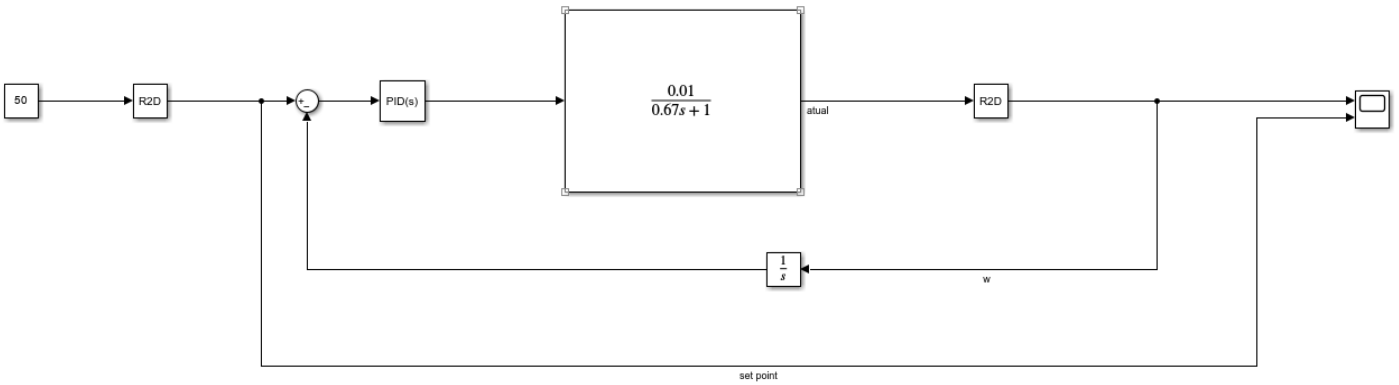

Figure 26 Transfer function of the block

Tunning

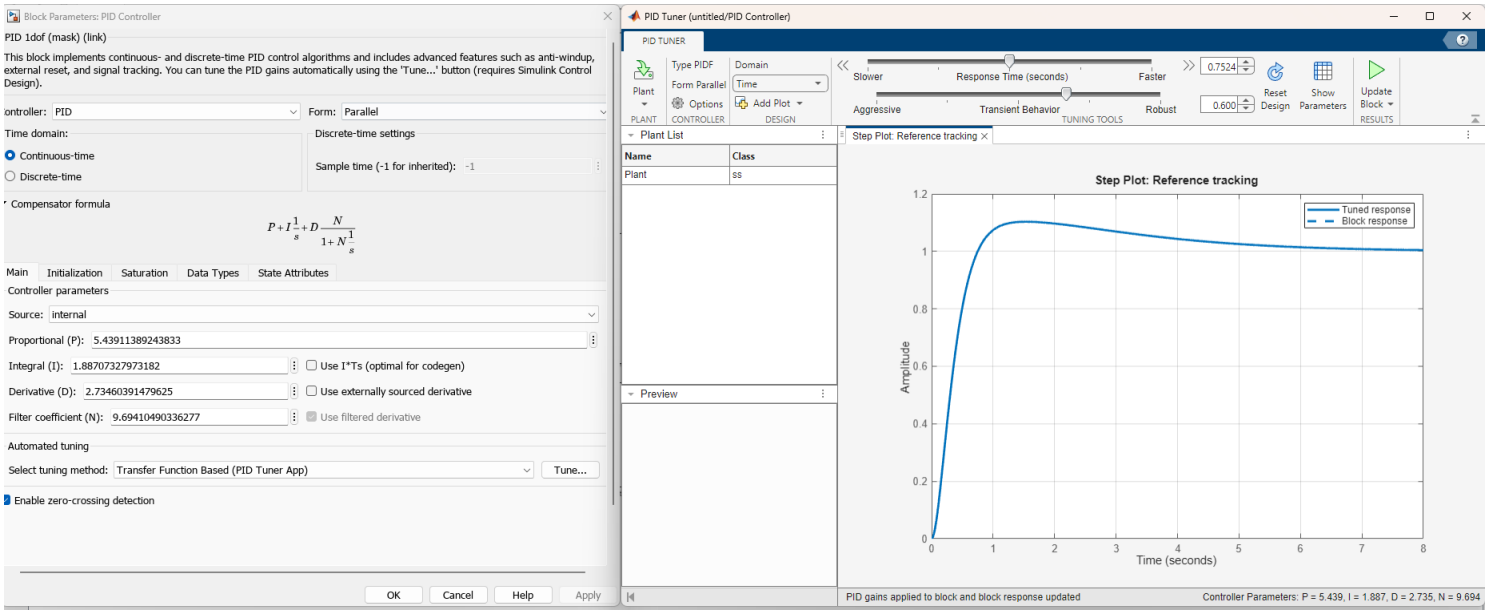

Figure 27 Tuning Prameters